

1. Abstract & Problem Definition

본 프로젝트는 Apache Kafka와 OpenMP를 활용한 고성능 보안 로그 분석 시스템을 구현하는 것을 목표로 한다. 시스템은 대량의 로그 메시지를 Kafka를 통해 스트리밍 방식으로 전달하고, Consumer에서 일정 크기의 배치 단위로 로그를 수집한 뒤 OpenMP 기반 병렬 처리를 통해 악성 패턴을 탐지한다.

대상 문제는 실시간 또는 준실시간 환경에서 발생하는 대량의 보안 로그 중 악성 행위를 빠르게 탐지하는 것이다. 예를 들어 본 시스템은 NOP sled 기반 버퍼 오버플로우 시도, format string payload, /etc/shadow 접근 시도, SQL injection, path traversal, XSS payload와 같은 의심스러운 문자열 패턴을 탐지한다. 이러한 로그 분석 작업은 입력 로그 수가 증가할수록 문자열 검색 연산이 반복적으로 수행되기 때문에 계산 비용이 커질 수 있다.

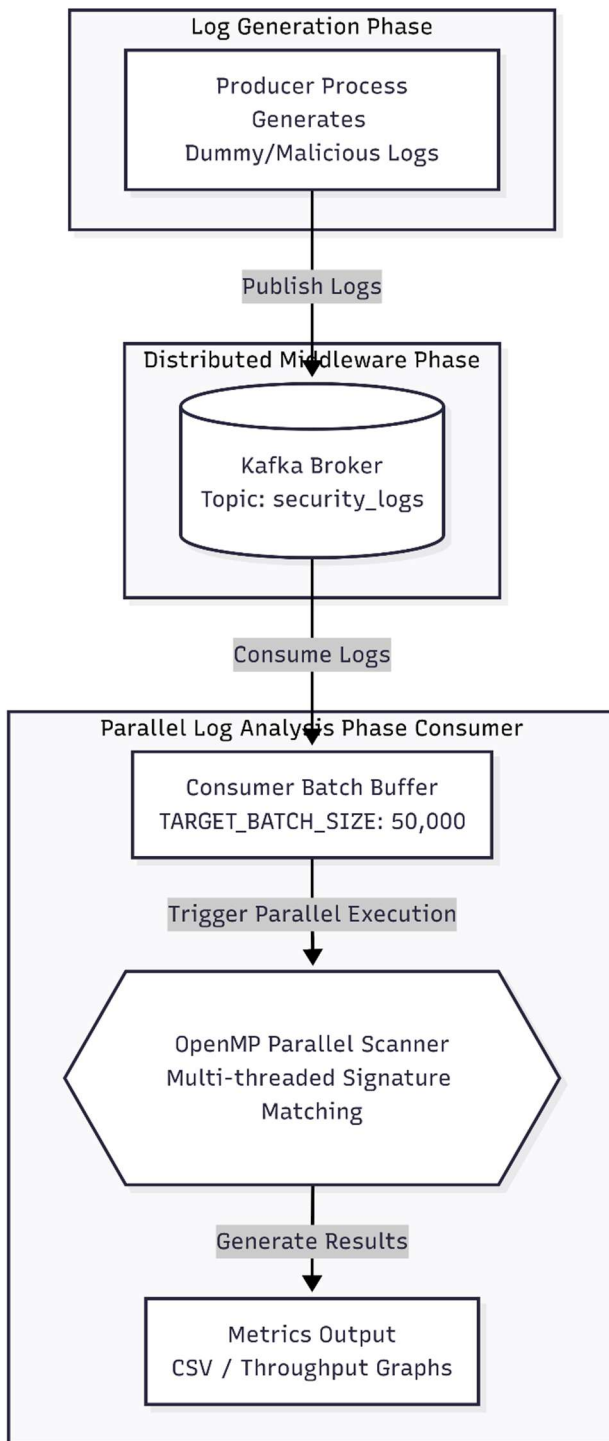
이 문제를 해결하기 위해 본 프로젝트는 분산 미들웨어와 병렬 컴퓨팅 기법을 결합하였다. Apache Kafka는 Producer와 Consumer를 분리하여 로그 생성 단계와 분석 단계를 독립적으로 동작하게 해 주며, 스트리밍 데이터 전달을 위한 안정적인 middleware 역할을 수행한다. OpenMP는 Consumer 내부의 배치 분석 단계에서 사용되며, 하나의 로그 배치를 여러 CPU thread가 나누어 처리함으로써 문자열 패턴 탐지 성능을 향상시킨다.

본 시스템의 주요 계산 병목은 Consumer에서 수행되는 batch 단위 signature matching 과정이다. 따라서 본 프로젝트에서는 OpenMP thread 수를 변화시키며 분석 처리량과 speedup을 측정하고, 병렬화가 로그 분석 단계의 성능 향상에 어떤 영향을 주는지 평가한다. 이를 통해 Kafka 기반 분산 통신 구조와 OpenMP 기반 병렬 처리가 하나의 end-to-end 로그 분석 파이프라인 안에서 어떻게 결합될 수 있는지 보인다.

2. System Architecture

본 시스템은 크게 Producer, Kafka middleware, Consumer, OpenMP 기반 병렬 분석 단계로 구성된다. Producer는 정상 로그와 악성 payload가 포함된 로그를 생성하여 Kafka topic으로 전송한다. Kafka는 Producer와 Consumer 사이에서 메시지를 저장하고 전달하는 distributed middleware 역할을 수행한다. Consumer는 Kafka topic으로부터 로그 메시지를 읽어 batch buffer에 저장한 뒤, batch 크기가 설정값에 도달하면 OpenMP를 이용해 병렬 signature scanning을 수행한다.

전체 데이터 흐름은 다음과 같다.



Producer와 Consumer는 서로 직접 통신하지 않고 Kafka topic을 통해 간접적으로 연결된다. 이러한 구조는 로그 생성 단계와 로그 분석 단계를 분리하여 각 구성 요소가 독립적으로 실행될 수 있게 한다. Producer는 로그를 지속적으로 Kafka에 publish하고, Consumer는 Kafka로부터 메시지를 consume하여 분석한다. 따라서 Producer의 로그 생성 속도와 Consumer의 분석 속도가 일시적으로 달라지더라도 Kafka가 중간 buffer 역할을 수행할 수 있다.

Kafka는 security_logs라는 topic을 사용하며, Docker Compose 환경에서 단일 broker로 실행된다. Topic은 4개의 partition으로 생성된다. 본 프로젝트는 학부 기준의 localhost 기반 실험 환경을 대상으로 하므로 다중 물리 머신 배포는 수행하지 않았다. 대신 Docker Compose를 사용하여 Kafka와 Zookeeper를 독립된 container로 실행하고, Producer와 Consumer는 host 환경에서 별도 process로 실행하였다.

Consumer 내부에서는 batch 기반 처리 구조를 사용한다. Consumer는 Kafka에서 메시지를 하나씩 읽어 log_batch에 저장하고, batch size에 도달하면 해당 batch를 분석 단계로 넘긴다. 분석 단계에서는 OpenMP의 parallel for를 사용하여 batch 안의 로그들을 여러 thread가 나누어 처리한다. 각 thread는 자신에게 할당된 로그에서 악성 signature를 검색하고, 탐지된 로그 수를 local하게 계산한다. 이후 OpenMP reduction을 통해 전체 threat count를 안전하게 합산한다.

Thread/process 조직은 다음과 같이 요약할 수 있다.

Component	Role
Producer Process	정상 로그와 악성 로그를 생성하고 Kafka topic으로 전송
Kafka Broker	로그 메시지를 저장하고 Consumer에게 전달하는 middleware 역할 수행
Consumer Process	Kafka에서 로그를 수신하고 batch 단위로 분석 수행
OpenMP Threads	Consumer 내부에서 batch 내 로그를 병렬로 signature scanning

이 구조를 통해 본 시스템은 명세서에서 요구하는 distributed middleware와 parallel acceleration을 하나의 end-to-end pipeline 안에 통합한다. Kafka는 분산 통신 및 구조적 decoupling을 담당하고, OpenMP는 Consumer 내부의 계산 병목인 로그 signature matching 단계를 병렬화하여 처리 성능을 향상시킨다.

2.1 Design Rationale

본 프로젝트에서는 로그 생성 단계와 로그 분석 단계를 분리하기 위해 Apache Kafka를 distributed middleware로 선택하였다. Kafka는 Producer와 Consumer가 직접 연결되지 않고 topic을 통해 데이터를 교환하도록 하므로, 두 구성 요소의 실행 속도가 달라져도 시스템 구조를 안정적으로 유지할 수 있다. 또한 로그 스트리밍 환경을 자연스럽게 모델링할 수 있어 고속 로그 분석 시스템에 적합하다.

병렬 처리 프레임워크로는 OpenMP를 선택하였다. Consumer는 Kafka로부터 수신한 로그를 batch 단위로 모은 뒤 각 로그에 대해 동일한 signature matching 작업을 반복한다. 이러한 작업은 로그 단위로 독립적이기 때문에 여러 CPU thread에 쉽게 분배할 수 있다. OpenMP는 C++ 코드에 비교적 적은 수정으로 parallel for와 reduction을 적용할 수 있으므로, 본 프로젝트의 batch 기반 로그 분석 단계에 적합하다. 따라서 Kafka는 distributed communication과 structural decoupling을 담당하고, OpenMP는 Consumer 내부의 computational bottleneck을 완화하는 역할을 담당한다. 이 조합은 명세서에서 요구하는 distributed middleware와 parallel acceleration을 단순하면서도 명확하게 통합할 수 있는 설계이다.

3. Implementation Details

본 프로젝트는 C++로 Producer와 Consumer를 구현하였으며, Kafka 연동을 위해 librdkafka 라이브러리를 사용하였다. 전체 실행 환경은 Docker Compose를 통해 Kafka와 Zookeeper를 실행하고, Producer와 Consumer는 host 환경에서 각각 별도 process로 실행하는 방식으로 구성하였다.

Producer는 정상 로그와 악성 로그 payload를 미리 정의된 문자열 목록에서 선택하여 Kafka topic인 security_logs로 전송한다. 정상 로그는 일반적인 HTTP 요청, 로그인 요청, 데이터베이스 연결 메시지 등으로 구성하였다. 악성 로그는 보안 위협 탐지를 실험하기 위해 NOP sled, format string payload, command injection, SQL injection, path traversal, XSS payload 등을 포함하도록 구성하였다. Producer는 --messages, --rate, --malicious-percent 옵션을 통해 생성할 메시지 수, 초당 전송률, 악성 로그 비율을 조절할 수 있다.

Kafka middleware는 Docker Compose를 통해 실행된다. 본 프로젝트에서는 단일 Kafka broker를 사용하며, security_logs topic은 4개의 partition으로 생성된다. Kafka는 Producer와 Consumer를 직접 연결하지 않고 topic을 통해 메시지를 전달함으로써 두 구성 요소를 구조적으로 분리한다. Consumer는 Kafka의 consumer group 기능을 사용하여 security_logs topic을 구독하고 로그 메시지를 수신한다.

Consumer는 Kafka로부터 수신한 로그를 batch buffer에 저장한다. Batch size는 실행 옵션으로 조절할 수 있으며, 실험에서는 batch size를 50,000으로 설정하였다. Batch buffer가 가득 차면 Consumer는 해당 batch를 OpenMP 기반 병렬 분석 단계로 전달한다. 이 방식은 메시지를 하나씩 분석하는 것보다 thread 병렬화에 적합하며, batch 단위로 처리량을 측정하기 쉽다는 장점이 있다.

로그 분석 단계에서는 미리 정의된 threat signature 목록을 사용한다. Consumer는 각 로그 문자열에 대해 다음과 같은 악성 패턴이 포함되어 있는지 검사한다.

Signature Type	Example Pattern
Buffer overflow indicator	NOP sled pattern
Format string attack	%x%x%x%x%n
Sensitive file access	/etc/shadow
SQL injection	OR '1'='1
Path traversal	../
XSS payload	<script>

병렬 처리는 OpenMP의 parallel for 지시어를 사용하여 구현하였다. Batch 안의 로그들은 여러 OpenMP thread에 분배되며, 각 thread는 자신에게 할당된 로그에 대해 signature matching을 수행한다. 탐지된 악성 로그 수는 thread별로 계산된 뒤 OpenMP reduction을 통해 하나의 total_threats 값으로 합산된다. 이 방식은 여러 thread가 동시에 같은 변수를 갱신할 때 발생할 수 있는 data race를 방지한다.

성능 평가의 안정성을 위해 work-factor 옵션을 추가하였다. 단순 문자열 검색은 매우 짧은 시간 안에 끝날 수 있기 때문에 thread 수에 따른 성능 차이가 측정 오버헤드에 묻힐 수 있다. 이를 보완하기 위해 work-factor는 각 로그에 대해 signature scanning을 반복 수행하는 횟수를 조절한다. 실험에서는 work-factor를 64로 설정하여 분석 단계의 계산량을 늘리고, OpenMP thread scaling 결과를 더 안정적으로 측정하였다.

Consumer는 각 batch 처리 후 탐지된 threat 수, 병렬 처리 시간, 처리량, 순차 실행 시간, speedup을 출력한다. 또한 --csv 옵션을 통해 결과를 CSV 파일로 저장할 수 있다. 본 실험에서는 CSV 결과를 Excel로 분석하여 thread 수에 따른 throughput 및 speedup 그래프를 작성하였다.

4. Experimental Evaluation

본 프로젝트에서는 OpenMP thread 수 변화에 따른 로그 분석 성능을 평가하였다. 실험의 목적은 Consumer 내부의 batch 분석 단계에서 thread 수가 증가할 때 throughput과 speedup이 어떻게 변화하는지 확인하는 것이다. 실험은 localhost 환경에서 Docker Compose로 Kafka와 Zookeeper를 실행한 뒤, Producer와 Consumer를 별도 process로 실행하여 수행하였다.

실험에서는 batch size를 50,000으로 고정하고, 각 thread 설정마다 5개의 batch를 처리하였다. OpenMP thread 수는 1, 2, 4, 8로 변경하였다. 또한 짧은 문자열 검색 연산으로 인해 측정 시간이 지나치게 짧아지는 것을 방지하기 위해 work-factor를 64로 설정하였다. 각 설정에서 Consumer는 batch 처리 시간, 처리량, 순차 실행 시간, speedup을 CSV 파일로 기록하였다.

실험 결과, OpenMP thread 수가 증가함에 따라 Consumer의 로그 분석 처리량이 증가하였다. 1 thread에서는 평균 281,121 logs/sec의 처리량을 보였고, 8 threads에서는 평균 877,821 logs/sec까지 증가하였다. Speedup 역시 1 thread 기준 1.00x에서 8 threads 기준 2.81x까지 증가하였다. 이는 batch 단위 로그 분석 작업이 OpenMP 병렬화에 의해 성능 향상을 얻을 수 있음을 보여준다.

다만 speedup은 thread 수에 비례하여 선형적으로 증가하지는 않았다. 1 thread에서 2 threads로 증가할 때는 큰 성능 향상이 나타났고, 4 threads에서도 추가적인 향상이 있었다. 그러나 4 threads에서 8 threads로 증가할 때의 speedup 향상 폭은 상대적으로 작았다. 이는 문자열 검색 기반 workload의 계산 강도가 제한적이며, thread 수가 증가할수록 thread scheduling overhead, memory access contention, cache 효과 등이 성능에 영향을 주기 때문으로 해석할 수 있다.

본 실험에서 측정한 throughput은 전체 Kafka pipeline의 end-to-end throughput이 아니라, Consumer가 batch를 수집한 이후 수행하는 OpenMP 분석 단계의 throughput이다. 즉, Kafka message ingestion, network overhead, producer 전송 속도 등은 해당 throughput 계산에 직접 포함되지 않는다. 본 실험의 초점은 distributed middleware와 병렬 분석 단계가 결합된 구조에서, Consumer 내부의 계산 병목이 OpenMP thread scaling에 의해 얼마나 개선되는지를 평가하는 것이다.

보고서에는 위 결과를 바탕으로 다음 두 개의 그래프를 포함하였다.

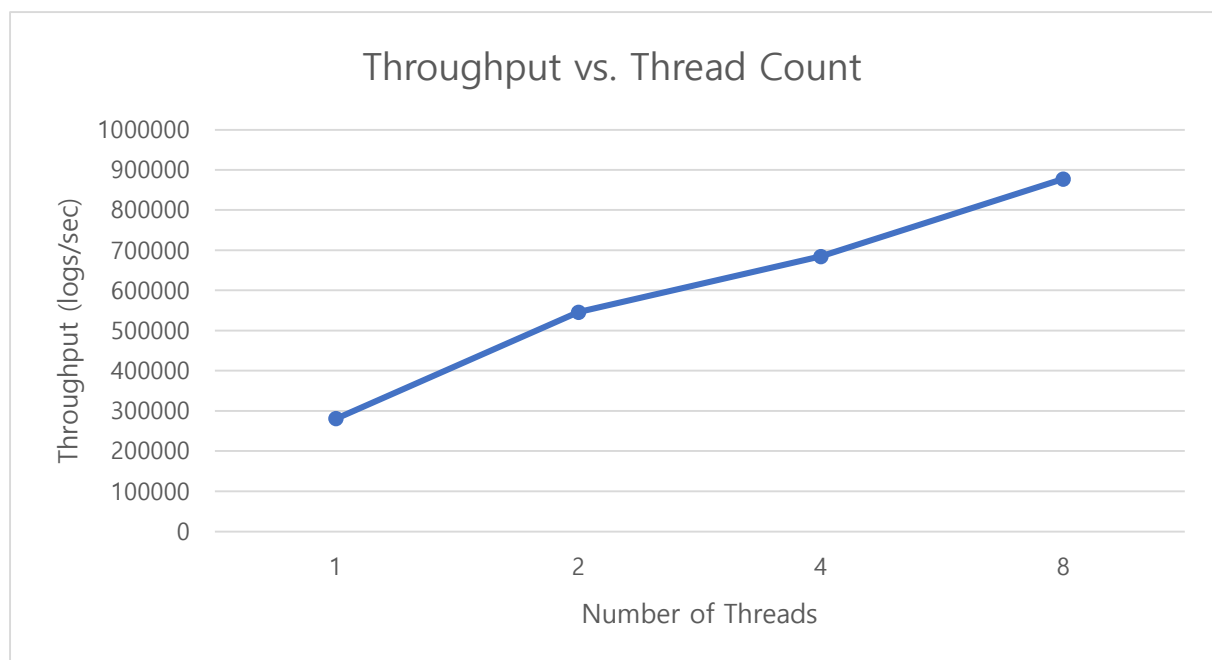


Figure 1 Throughput vs. Number of OpenMP Threads

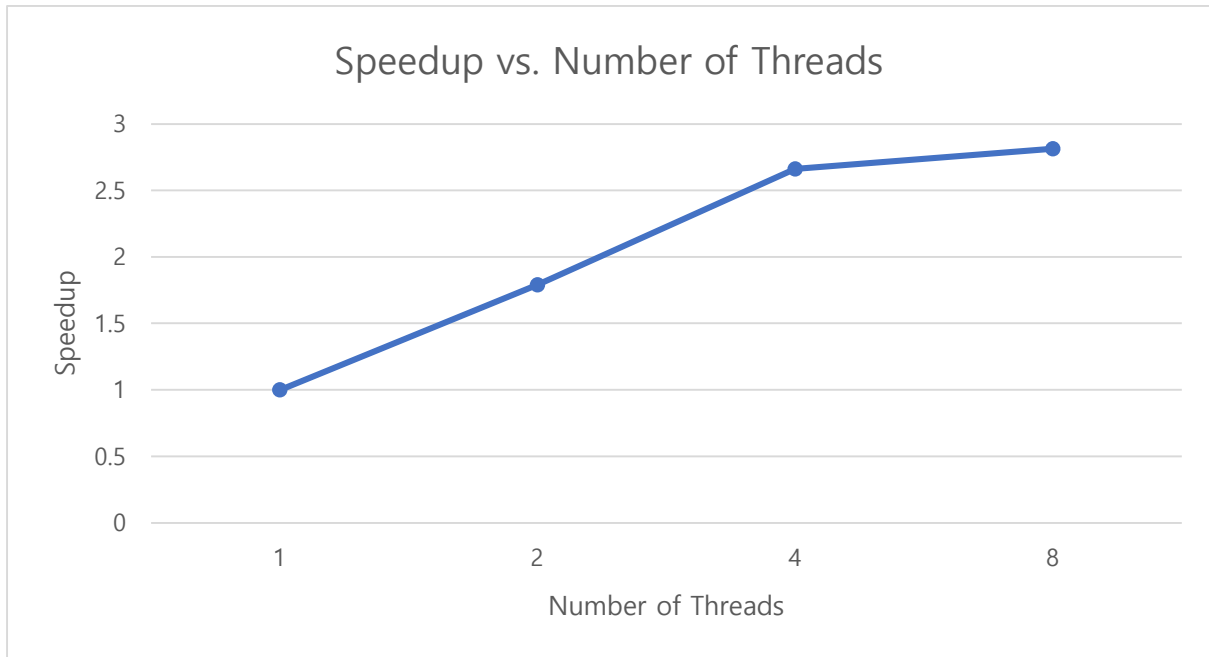


Figure 2 Speedup vs. Number of OpenMP Threads

Figure 1은 thread 수 증가에 따른 평균 throughput 변화를 보여준다. Thread 수가 1에서 8로 증가함에 따라 throughput이 전반적으로 증가하는 경향을 확인할 수 있다. Figure 2는 1 thread 실행을 기준으로 계산한 speedup을 보여준다. Speedup은 8 threads에서 2.81x까지 증가하였으나, thread 수가 증가할수록 증가폭은 감소하였다. 이는 병렬화에 따른 성능 향상과 동시에 병렬 처리 overhead가 존재함을 보여준다.

5. Bottleneck Analysis

본 시스템의 주요 병목은 Consumer 내부의 batch 기반 signature matching 단계에서 발생한다. Producer는 로그 메시지를 생성하여 Kafka에 전송하고, Kafka는 이를 topic에 저장한 뒤 Consumer에게 전달한다. 그러나 실제 보안 로그 분석에서 가장 많은 계산이 필요한 부분은 각 로그 문자열에 대해 여러 threat signature를 반복적으로 비교하는 과정이다. 따라서 본 프로젝트에서는 이 분석 단계를 OpenMP로 병렬화하여 병목을 완화하고자 하였다.

실험 결과 OpenMP thread 수가 증가함에 따라 throughput과 speedup이 향상되었지만, 성능 향상은 선형적으로 증가하지 않았다. 1 thread에서 2 threads로 증가할 때는 speedup이 1.79x로 크게 향상되었고, 4 threads에서는 2.66x까지 증가하였다. 그러나 8 threads에서는 2.81x로, 4 threads 대비 증가 폭이 상대적으로 작았다. 이는 thread 수가 증가할수록 병렬 처리로 얻는 이득뿐만 아니라 thread scheduling overhead와 memory/cache contention도 함께 증가하기 때문이다.

특히 본 프로젝트의 분석 작업은 문자열 검색 기반 workload이다. 문자열 검색은 행렬 연산이나 수치 시뮬레이션처럼 계산량이 매우 큰 작업에 비해 각 데이터 항목당 수행되는 연산량이 제한적이다. 따라서 thread 수를 과도하게 늘리면 각 thread가 처리하는 작업량은 줄어들지만, thread 관리 비용과 메모리 접근 비용이 상대적으로 커질 수 있다. 이러한 이유로 4 threads 이후에는 speedup 증가 폭이 둔화된 것으로 볼 수 있다.

Batch size 또한 중요한 성능 요소이다. Batch size가 너무 작으면 OpenMP 병렬 영역을 생성하고 thread를 동기화하는 비용이 실제 분석 시간에 비해 커져 병렬화 효율이 낮아질 수 있다. 반대로 batch size가 너무 크면 throughput은 좋아질 수 있지만, batch가 가득 찰 때까지 기다리는 시간이 길어져 latency가 증가할 수 있다. 본 실험에서는 batch size를 50,000으로 설정하여 thread scaling을 비교적 안정적으로 측정하였다.

Kafka middleware 측면에서는 단일 broker와 localhost 환경이 한계로 작용할 수 있다. 본 프로젝트는 학부 기준의 단일 머신 Docker Compose 환경에서 수행되었기 때문에, 실제 다중 노드 Kafka cluster에서 발생할 수 있는 network overhead, broker replication overhead, partition rebalance 등의 요소는 평가하지 않았다. 따라서 본 실험 결과는 production 환경의 Kafka cluster 성능을 나타내기보다는, Kafka를 사용한 distributed middleware 구조와 OpenMP 기반 병렬 분석 단계의 통합 가능성을 보여주는 데 초점을 둔다.

또한 본 실험에서 측정한 throughput은 전체 end-to-end pipeline throughput이 아니라 Consumer 내부의 OpenMP 분석 단계 throughput이다. Kafka에서 메시지를 consume하는 시간, Producer의 메시지 생성 속도, broker queueing 시간은 throughput 계산에 포함되지 않았다. 따라서 전체 시스템 성능을 더 정밀하게 분석하려면 추후 end-to-end latency 측정, Kafka consumer throughput 측정, batch size scaling 실험 등을 추가할 수 있다.

결론적으로 본 시스템에서 성능에 영향을 주는 주요 요소는 OpenMP thread 수, batch size, 문자열 검색 workload의 계산 강도, Kafka broker 구성이다. 본 프로젝트에서는 OpenMP 병렬화를 통해 Consumer 분석 단계의 처리량을 유의미하게 향상시켰지만, thread 수 증가에 따른 overhead로 인해 speedup은 제한적으로 증가하였다. 이는 병렬 시스템 설계에서 단순히 thread 수를 늘리는 것보다 workload 특성과 batch 크기에 맞는 적절한 병렬화 설정이 중요함을 보여준다.

6. Conclusion

본 프로젝트에서는 Apache Kafka와 OpenMP를 결합한 고성능 보안 로그 분석 시스템을 설계하고 구현하였다. Kafka는 로그 Producer와 Consumer를 분리하는 distributed

middleware 역할을 수행하였고, OpenMP는 Consumer 내부의 batch 기반 signature matching 단계를 병렬화하는 데 사용되었다. 이를 통해 본 시스템은 명세서에서 요구하는 distributed communication과 parallel acceleration을 하나의 end-to-end pipeline 안에 통합하였다.

구현된 시스템은 Producer가 정상 로그와 악성 payload가 포함된 로그를 생성하여 Kafka topic으로 전송하고, Consumer가 이를 batch 단위로 수집한 뒤 OpenMP thread를 이용해 병렬 분석하는 구조를 가진다. Consumer는 NOP sled, format string attack, SQL injection, path traversal, XSS payload 등 여러 보안 위협 signature를 검사하며, 분석 결과로 threat count, processing time, throughput, speedup을 출력한다.

성능 평가에서는 OpenMP thread 수를 1, 2, 4, 8로 변화시키며 thread count scaling 실험을 수행하였다. Batch size는 50,000, work factor는 64로 설정하였고, 각 thread 설정마다 5개의 batch를 처리하였다. 실험 결과, 분석 단계 throughput은 1 thread에서 평균 281,121 logs/sec, 8 threads에서 평균 877,821 logs/sec로 증가하였다. Speedup은 1 thread 대비 8 threads에서 약 2.81x를 달성하였다.

이 결과는 batch 기반 로그 분석 작업이 OpenMP 병렬화에 의해 성능 향상을 얻을 수 있음을 보여준다. 다만 speedup은 thread 수에 비례하여 선형적으로 증가하지 않았으며, 4 threads 이후에는 성능 향상 폭이 감소하였다. 이는 문자열 검색 workload의 제한된 계산 강도, thread scheduling overhead, memory/cache contention 등의 영향으로 해석할 수 있다.

본 프로젝트는 Kafka + OpenMP 조합을 활용하여 distributed middleware와 parallel computing framework를 올바르게 통합하였다. 또한 thread count scaling 실험과 throughput/speedup 그래프를 통해 기본적인 scalability analysis를 수행하였다. 향후 개선 방향으로 end-to-end latency 측정, batch size scaling 실험, 다중 Consumer를 이용한 worker scaling, 또는 다중 broker Kafka cluster 환경에서의 실험을 고려할 수 있다.

결론적으로 본 시스템은 대량의 로그 스트림을 Kafka로 전달하고, Consumer 내부에서 OpenMP를 이용해 병렬 분석함으로써 로그 분석 처리량을 향상시킬 수 있음을 확인하였다. 이를 통해 분산 미들웨어와 병렬 처리 기법을 결합한 고성능 데이터 처리 파이프라인의 설계 및 구현 가능성을 보였다.